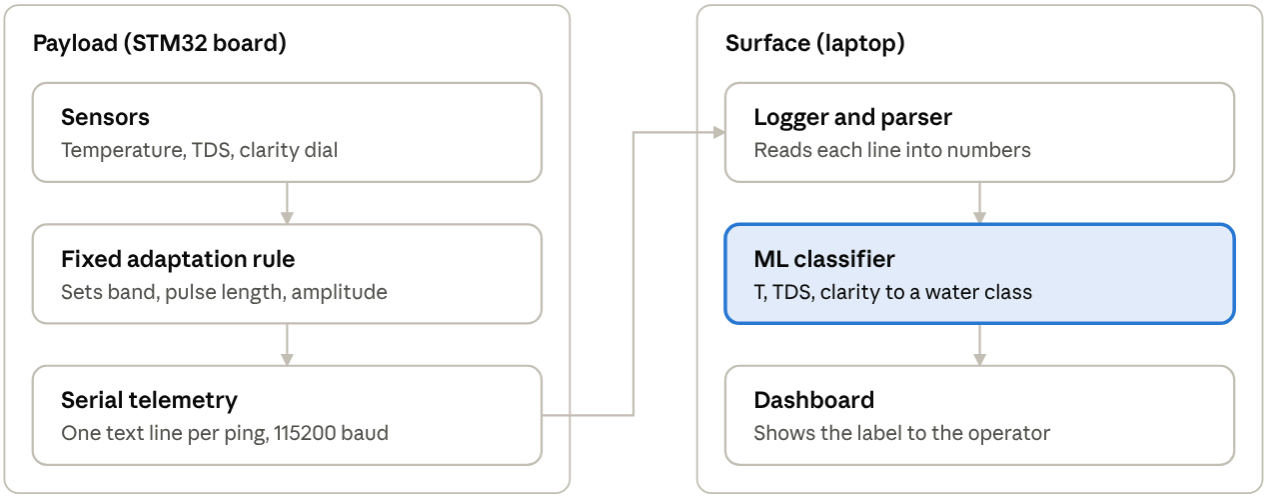
DHVANI Water-Condition ML Model – How to Build It

Oct 3, 2026 · @harshit

What the model does

The model runs on the laptop, not on the board. It reads the numbers the DHVANI board prints every ping and labels the water type for the operator. The board itself keeps using a simple, fixed rule to choose each ping — that keeps it fast and low-power.

The ML model labels the water on the laptop; the payload never waits for it



where the model runs · payload rule vs surface ML

In one line: **sensor readings in, "what kind of water is this?" out.** The board never waits for the model; if the laptop is unplugged, the payload works exactly the same.

Inputs and the four water classes

The model uses only the three readings the board actually sends, and predicts one of four water types named after the brief's own scenarios.

Inputs (features):

Feature	From serial field	Range	Real or simulated
`temp_c`	`T=`	about 15–40 °C	Real (DS18B20 probe)
`tds_ppm`	`tds=`	0–1500+ ppm	Real (TDS probe)

Feature	From serial field	Range	Real or simulated
`clarity`	`clarity=`	0 (muddy) – 4095 (clear)	Simulated (potentiometer dial)

Do **not** use the old `turbidity_v` or `depth_norm` columns — the board no longer has those sensors.

Output (the four classes):

Class	Clarity dial	Water
Clear freshwater	Left (clear)	Plain tap water
Clear shallow reef	Left (clear)	Salty water
Muddy river	Right (muddy)	Plain tap water
Muddy estuary	Right (muddy)	Salty water

Why these four: they use **two** real-world dimensions (clarity and salt), so the model has to combine inputs — and "Clear Shallow Reef" and "Muddy Estuary" are the exact scenario names in the PS 26058 brief.

If you would rather keep the original four turbidity levels (Clear / Slightly / Moderately turbid / Murky), the steps below still work — just label by dial position only.

Step 1 — Collect data from the board

Record real serial lines while the board sits in each water condition, and save them to a CSV with the class and a session number.

Setup: board running (RESET first, so it stays NORMAL), TDS and temperature probes in a glass, laptop on the STLink COM port. Close the CubeIDE serial console first — only one program can hold the port.

Recording plan: 3 sessions per class, about 3 minutes each (≈ 700 rows per session). Between sessions, rinse the probes, change the glass, and move the dial a little within its zone, so sessions differ the way real dives would.

Class	Dial	Glass
clear_fresh	Left quarter	Fresh tap water
clear_reef	Left quarter	Tap water + 1 spoon salt
muddy_river	Right quarter	Fresh tap water

Class	Dial	Glass
muddy_estuary	Right quarter	Tap water + 1 spoon salt

Logger script (`logger.py`) — run once per session, e.g. `python logger.py COM9 clear_reef 2`:

```
import csv, re, sys, time, serial

port, label, session = sys.argv[1], sys.argv[2], int(sys.argv[3])
LINE = re.compile(r"T=(-?[\d.]+)C c=(\d+) clarity=(\d+) .*?tds=(\d+)ppm")

with serial.Serial(port, 115200, timeout=2) as ser, \
     open("dhvani_data.csv", "a", newline="") as f:
    w = csv.writer(f)
    if f.tell() == 0:
        w.writerow(["time", "session", "label", "temp_c", "tds_ppm",
"clarity"])
    end = time.time() + 180                    # 3 minutes
    while time.time() < end:
        m = LINE.search(ser.readline().decode(errors="ignore"))
        if m:                                   # skip partial lines
            w.writerow([time.time(), f"{label}-{session}", label,
                        float(m.group(1)), int(m.group(4)), int(m.group(3))])
```

Install once: `pip install pyserial pandas scikit-learn joblib`.

Step 2 — Label the data honestly

Each row's label is the condition you physically set up during that recording session — not a number computed from the inputs.

- **Why it matters:** the old notebook scored $R^2 = 0.99995$ because its target was almost certainly a formula of the inputs, so the forest just re-learned the formula. A judge will ask how labels were made; "we set up the water and recorded it" is the answer that holds up.
- **The logger already writes the label** from the command line, so labels come for free.
- **Throw away the first ~10 seconds** of each session (the averaging filter on the board is still settling), and any rows where `tds_ppm` is near 0 (probe out of the water).
- **Be upfront about the dial:** clarity is simulated by a potentiometer, so the clarity feature is clean by design. The real signal the model must learn is the salt (TDS) and how it combines with clarity.

Quick clean-up in pandas:

```
import pandas as pd
df = pd.read_csv("dhvani_data.csv")
df = df[df.tds_ppm > 5]                                  # probe in water
df = df[df.groupby("session").cumcount() >= 40]      # drop each session's
settling rows
print(df.label.value_counts())                            # check classes are
balanced
```

Step 3 — Train the model

Train a small classifier on sessions 1–2 of every class and keep session 3 completely unseen for testing.

- **Model:** a Random Forest classifier with 20 trees, max depth 5 — many small decision trees that vote. Small is enough for three inputs, trains in seconds, and stays easy to explain.
- **Split by session, not randomly:** neighbouring rows are near-copies, so a random split leaks test rows into training and inflates the score. Holding out whole sessions tests the model on a recording it has never seen.

`train.py`:

```
import pandas as pd, joblib
from sklearn.ensemble import RandomForestClassifier

FEATURES = ["temp_c", "tds_ppm", "clarity"]

df = pd.read_csv("dhvani_data.csv")
df = df[df.tds_ppm > 5]
df = df[df.groupby("session").cumcount() >= 40]      # drop each session's
settling rows

test_mask = df.session.str.endswith("-3")      # session 3 of each class =
unseen
train, test = df[~test_mask], df[test_mask]

model = RandomForestClassifier(n_estimators=20, max_depth=5, random_state=0)
model.fit(train[FEATURES], train.label)

joblib.dump(model, "dhvani_model.joblib")
test.to_csv("test_rows.csv", index=False)
print("trained on", len(train), "rows; testing on", len(test))
```

If temperature turns out to matter a lot, check whether one class was simply recorded in warmer water — that is a recording accident, not physics. Record more evenly or drop `temp_c`.

Step 4 — Evaluate it fairly

Report accuracy on the unseen sessions, a confusion matrix, and a simple-rule baseline the model has to beat or match.

`evaluate.py`:

```
import pandas as pd, joblib
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

FEATURES = ["temp_c", "tds_ppm", "clarity"]
model = joblib.load("dhvani_model.joblib")
test = pd.read_csv("test_rows.csv")

pred = model.predict(test[FEATURES])
print("Model accuracy:", accuracy_score(test.label, pred))
print(confusion_matrix(test.label, pred, labels=model.classes_))
print(classification_report(test.label, pred))

# Baseline: two hand-written thresholds, no ML
def rule(r):
    clear = r.clarity > 2048
    salty = r.tds_ppm > 400          # set from your own fresh vs salty
readings
    return {(True, False): "clear_fresh", (True, True): "clear_reef",
            (False, False): "muddy_river", (False, True): "muddy_estuary"}
[(clear, salty)]
base = test.apply(rule, axis=1)
print("Baseline accuracy:", accuracy_score(test.label, base))
```

How to read the results:

You see	What it means	What to say
Model ≈ baseline, both high	The classes are cleanly separable by two thresholds	"A simple rule already works; the model matches it and generalises to unseen sessions."

You see	What it means	What to say
Model clearly beats baseline	It found boundaries the hand thresholds missed (e.g. TDS drift with temperature)	Quote both numbers
Confusion between clear_fresh and clear_reef	Salt levels too close between sessions	Use more salt, or record more sessions
Accuracy near 100% on the first try	Possible leakage	Re-check that test sessions were never used in training

Always quote the **session-split** number, never a random-split one.

Step 5 — Use it in the dashboard

The dashboard loads the saved model once, then classifies every new serial line in real time.

```
import re, joblib, pandas as pd, serial

FEATURES = ["temp_c", "tds_ppm", "clarity"]
NAMES = {"clear_fresh": "Clear freshwater", "clear_reef": "Clear shallow
reef",
         "muddy_river": "Muddy river", "muddy_estuary": "Muddy estuary"}
LINE = re.compile(r"T=(-?[\d.]+)C c=(\d+) clarity=(\d+) .*?tds=(\d+)ppm")
model = joblib.load("dhvani_model.joblib")

with serial.Serial("COM9", 115200, timeout=2) as ser:   # listen only, never
write
    while True:
        m = LINE.search(ser.readline().decode(errors="ignore"))
        if not m:
            continue
        x = pd.DataFrame([[float(m.group(1)), int(m.group(4)),
int(m.group(3))]], columns=FEATURES)
        label = model.predict(x)[0]
        conf = model.predict_proba(x).max()
        print(f"{NAMES[label]}  ({conf:.0%} sure)")   # → the big panel on
the dashboard
```

- Put the label in the **big coloured panel** of the dashboard, with the confidence underneath.
- **Smooth it:** show the most common label of the last 5 pings, so the panel doesn't flicker when the dial is between zones.

- The rest of the dashboard (sensors, waveform choice, power) comes from the same parsed line — see the Ground-Station Dashboard build spec.

Checklist and what to report

When every box below is ticked, the model is ready for the dashboard, the video and the slide.

- [ ] 12 sessions recorded (4 classes × 3 sessions) into `dhvani_data.csv`
- [ ] Labels come from the recorded condition, not a formula
- [ ] Settling rows and probe-out-of-water rows removed
- [ ] Trained on sessions 1–2, tested on session 3
- [ ] Session-split accuracy, confusion matrix and baseline accuracy written down
- [ ] `dhvani_model.joblib` saved and loading in the dashboard
- [ ] Dashboard label smoothed over the last 5 pings
- [ ] Screen recording of the dashboard changing as the dial turns and salt is added

For the slide (fill in your numbers):

> "A Random Forest classifier on the ground station labels the water as clear freshwater, clear shallow reef, muddy river or muddy estuary from live payload telemetry. Trained on recorded sessions and tested on unseen ones: __% accuracy (simple two-threshold rule: __%). Clarity is simulated by a dial; temperature and salinity are measured."

If a judge asks "why not run it on the chip?" — the payload's job is to choose a ping fast and cheaply, which a fixed rule does in microseconds at no extra power. The model's job is to help the person planning the mission, so it runs where that person is.